

Machine Learning for Physicists

A Nutshell Guide to the Practical Man

Caio Laganá Fernandes

Preface

So, you're a physicist looking to dive into the world of machine learning. You have a deep understanding of calculus, are familiar with experimental data-fitting methods, and have mastered concepts like entropy, non-linearity and hypersurface—yet you have only a vague idea of how a machine learning algorithm is actually written and how it works? Then this book is for you. And the good news is: your learning curve will grow fast. I've been in your shoes. I used to view machine learning as a distant horizon, but I soon realized that the knowledge I'd gained in physics made it incredibly easy to quickly master the concepts and engineering behind the field. So, come along and be amazed.

Chapter 1

One-neuron ML

You have been using a single-neuron network since the first time you fitted experimental data, but perhaps no one ever told you that. Let's recall the basics. You have a set of N experimental data points

$$(x_i, y_i) \tag{1.1}$$

through which to fit a linear function

$$f(x) = ax + b. \tag{1.2}$$

The Mean Squared Error Loss, defined as

$$\begin{aligned} L_{\text{MSE}}(a, b) &= \frac{1}{N} \sum_{i=1}^N [y_i - f(x_i)]^2 \\ &= \frac{1}{N} \sum_{i=1}^N [y_i - (ax_i + b)]^2 \end{aligned} \tag{1.3}$$

tells us how distant the curve $f(x)$ is from the datapoints. Fitting $f(x)$ to the datapoints (x_i, y_i) means minimizing $L_{\text{MSE}}(a, b)$ with respect to a and b , that is,

$$\frac{\partial}{\partial(a, b)} L_{\text{MSE}}(a, b) = 0. \tag{1.4}$$

No surprises so far, right? Well, what if I tell you the above is the simplest possible neural network? Indeed, it is a single-neuron network, where (1.2) are the model's weights, (1.3) is the loss function and (1.4) is the embryo of the gradient descent method.

Solving (1.4) for a and b , which, in this simple case is possible analytically, fixes the parameters to a_0 and b_0

$$\begin{aligned} a_0 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ b_0 &= \bar{y} - a_0 \bar{x} \end{aligned} \tag{1.5}$$

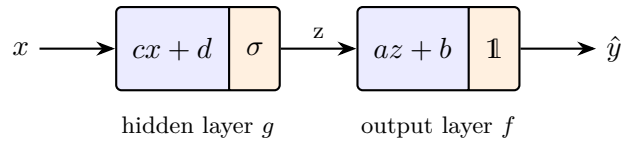
and we end up with a model $f_0(x) = a_0x + b_0$ (in (1.5), $\bar{x}$ and $\bar{y}$ are the mean values of the x_i and y_i datapoints). This model is a single neuron, no hidden layers: it receives one input parameter and outputs a number. f_0 has learned from data (1.1) through (1.4) and can now predict on any given x . Pretty cool, huh? The concepts were sitting there all the time. Take a moment to re-read the definitions and let's jump to the second simplest model: a two-neuron network.

Chapter 2

Two-neuron ML

Our neuron from Chap. 1 was of the form $f(x) = ax + b$ (always think of it as receiving one value x and, upon training for a, b parameters, outputting another value $a_0x + b_0$). Let us now place a second neuron, g , before it, so that g receives the input and f produces the final output. This new neuron will be of the form $g(x) = \sigma(cx + d)$, where $\sigma(x) = 1/(1 + e^{-x})$. The need for σ will be elucidated later; for now, just accept it as a magical ingredient. In fact, it is the neuron's so-called activation function!

Placing one neuron before another means composing the two functions, that is, given input x , the network outputs $f(g(x))$. In other words, input x feeds a (hidden) layer g , which applies its weight c and bias d , activates through σ and feeds output layer f , which in turn applies weight a and bias b and outputs $\hat{y}$. Layer f has identity operator $\mathbf{1}$ as its activation function. A visual diagram may help.



Our task now is to optimize the network for parameters a, b, c, d over training data. The loss function is similar to (1.3), just generalized for the two-neuron case,

$$L_{\text{MSE}}(a, b, c, d) = \frac{1}{N} \sum_{i=1}^N [y_i - f(g(x_i))]^2. \quad (2.1)$$

This time, however, MSE minimization, stated as

$$\frac{\partial}{\partial(a, b, c, d)} L_{\text{MSE}}(a, b, c, d) = 0 \quad (2.2)$$

cannot be solved analytically as in (1.5), and numerical methods must be used. The recipe is as follows: start at a certain initial position (a, b, c, d) in the four-dimensional parameter space,

then slightly change each coordinate by an amount according to¹:

$$\begin{aligned}
 a &\rightarrow a - \alpha \frac{\partial L_{\text{MSE}}}{\partial a} \\
 b &\rightarrow b - \alpha \frac{\partial L_{\text{MSE}}}{\partial b} \\
 c &\rightarrow c - \alpha \frac{\partial L_{\text{MSE}}}{\partial c} \\
 d &\rightarrow d - \alpha \frac{\partial L_{\text{MSE}}}{\partial d}
 \end{aligned} \tag{2.3}$$

The term α is called the learning rate (hallelujah, a name you’ve heard before, now in a clear context!). The derivatives of L_{MSE} in (2.3) point to the max uphill on the hypersurface, so the minus sign will move the initial coordinate to a new position where L_{MSE} is lower. After repeating this procedure a couple of times, (2.2) will be solved to a certain desired precision (yes, I am oversimplifying: this step is a billion-dollar industry).

The recipe just described for adjusting (a, b, c, d) parameters based on the partial derivatives (2.3) is called *gradient descent*. The computation of the derivatives themselves will lead to the concept of *backpropagation*. Stated like this, with bare partial derivatives and almost no intuition of what is going on behind the scenes, the recipe sounds a bit barren. Let me try to improve it.

The central idea of gradient descent and backpropagation is to tell (or, more precisely, to *signal*) the neurons in which direction they must adjust their parameters, and by how much, in order to minimize the final loss. Think about it: how can a hidden neuron (in our two-neuron model, this is g) possibly gather information about the loss at the output? This is the central problem to be addressed. Let’s dive a little into the algebra.

The partial derivatives present in (2.3), obtained through chain-rule derivatives on (2.1), can, in this case, be taken explicitly:

$$\begin{aligned}
 \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a} = -\frac{2}{N} \sum_i [y_i - f(g(x_i))] g(x_i) \\
 \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial b} = -\frac{2}{N} \sum_i [y_i - f(g(x_i))] \\
 \frac{\partial L}{\partial c} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial c} = -\frac{2a}{N} \sum_i [y_i - f(g(x_i))] \sigma'(cx_i + d) x_i \\
 \frac{\partial L}{\partial d} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial d} = -\frac{2a}{N} \sum_i [y_i - f(g(x_i))] \sigma'(cx_i + d)
 \end{aligned} \tag{2.4}$$

A suitable reparametrization of variables will be of great algebraic assistance. (We know from other branches of physics—electromagnetism, condensed matter, particle physics—that a “suitable change of variables” often carries deep phenomenological significance. So, hold on!)

$$\begin{array}{ll}
 x_i & \text{input} \\
 u_i = cx_i + d & \text{hidden pre-activation} \\
 z_i = \sigma(u_i) & \text{hidden activation} \\
 \hat{y}_i = f(g(x_i)) = az_i + b & \text{network output}
 \end{array} \tag{2.5}$$

In terms of (2.5), define the *error signal* as

$$\delta_i^{(2)} = \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{N} (y_i - \hat{y}_i), \tag{2.6}$$

¹Picture this process as finding a minimum on a graph. For reference, in Chap. 1 the graph of the loss function $(a, b, L_{\text{MSE}}(a, b))$ was a two-dimensional paraboloid embedded in $\mathbb{R}^3$; in this two-neuron scenario, the graph is a four-dimensional hypersurface embedded in $\mathbb{R}^5$. The paraboloid has a clear, well-defined minimum; this time, there may be several local minima plus saddle points and plateaus, and which one you land in depends on the starting point.

and the *hidden error signal* as

$$\delta_i^{(1)} = \delta_i a \sigma'(u_i). \quad (2.7)$$

As the name suggests, the error signals will *signal* the neurons in which direction to change their parameters, and by how much. With all these new variables, the partials (2.4) read

$$\begin{aligned} \frac{\partial L}{\partial a} &= \sum_i \delta_i^{(2)} z_i \\ \frac{\partial L}{\partial b} &= \sum_i \delta_i^{(2)} \\ \frac{\partial L}{\partial c} &= \sum_i \delta_i^{(1)} x_i \\ \frac{\partial L}{\partial d} &= \sum_i \delta_i^{(1)} \end{aligned} \quad (2.8)$$

Phew! Much cleaner than (2.4)! And in return we've gained an astonishing behaviour: instead of computing all the messy partial derivatives in (2.4), all we need to do in order to calibrate each neuron's parameters is to compute its respective error signal and apply elementary algebra. This shortcut is called *backpropagation*. The name stems from the fact that, starting at the output and going backwards, each crossed layer will render the neuron one error signal. Look carefully: a and b each gained a $\delta_i^{(2)}$ plus z_i for a ; similarly, c and d each gained a $\delta_i^{(1)}$ plus x_i for c . Were there a third layer, the recipe would repeat.

This behaviour, shown up explicitly in this two-neuron toy model, is in fact a general rule to networks of any size. This prepares our terrain for the next chapter.

Chapter 3

Multi-neuron ML

We now have all necessary ingredients to build a multi-neuron network. So, let's do it! The first step is to adapt the input layer to be able to receive, instead of a single scalar x , a more generic object. You may argue “the most generic object is an $N \times M$ matrix, or a tensor of arbitrary rank!” And you're right, as I thought I was the first time I philosophized about the subject. However... When you stop to think about it, you realize that any generic object, whether it is a matrix or a tensor, can be *flattened* to a vector of some dimension. This flattening process can always take place outside of the network, in a data preprocessing stage. So, as far as the network itself is concerned, the most general input object is an N -dimensional vector.

The above considerations lead us to define the architecture of the first layer as an N -neuron input layer. I will make use of tensor index notation to write the input parameter of (2.5) as

$$x_\nu^i \quad \text{input} \quad (3.1)$$

I have promoted the i -index to the top (remember, this is the index labeling the i th data point) while using greek letters at the bottom as the vector indices. With this notation, the hidden pre-activation step reads

$$u_\mu^i = w_{\mu\nu}^{(1)} x_\nu^i + b_\mu^{(1)} \quad \text{hidden pre-activation} \quad (3.2)$$

In (3.2), the weight c from (2.5) has turned into the $w_{\mu\nu}^{(1)}$ matrix (the letter w is the standard one for referring to the weights; I've been using a , b , c and d out of laziness) whereas the bias parameter, called d in (2.5), has turned into a bias vector $b_\mu^{(1)}$. The (1) superscript indicates these are the weights and bias of the first layer; as we start composing several layers, this number will grow. One very important observation: the ν -index ranges from 1 to the dimension of the input vector; the μ -index ranges from 1 to the size of the hidden layer. The $w_{\mu\nu}^{(1)}$ matrix connects every input scalar to every neuron in the hidden layer.

We must now define the activation function for the neurons in the hidden layer. Following nomenclature in equation (2.5) and plugging layer index (1) and neuron index μ , it becomes:

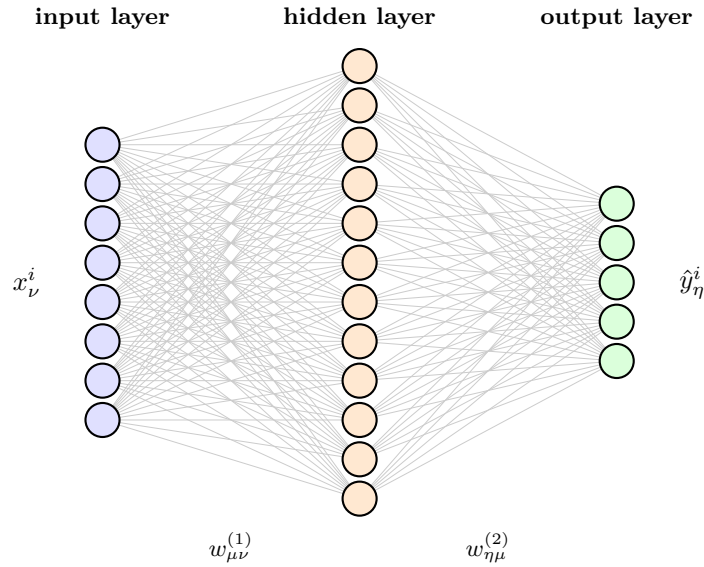
$$z_\mu^i = \sigma^{(1)}(u_\mu^i) \quad \text{hidden activation} \quad (3.3)$$

There are several options on the market for implementing $\sigma^{(1)}$ as an actual function, all of which are listed in Appendix A. The standard choice for hidden layers is ReLU, although this may vary from one problem to another. For now, I will leave all options open so that, in the coming chapters, we can select them and build actual models that operate on specific, concrete data. That said, we can move on to the final step: defining the output layer. Let's carry out both steps at once:

$$\begin{aligned} v_\eta^i &= w_{\eta\mu}^{(2)} z_\mu^i + b_\eta^{(2)} && \text{output pre-activation} \\ \hat{y}_\eta^i &= \sigma^{(2)}(v_\eta^i) && \text{network output} \end{aligned} \quad (3.4)$$

A few words on (3.4) are worth saying. The index η runs over the number of neurons in the output layer. It can be one neuron with linear activation, if we are talking about a problem that must output one single unbounded number; it can be two neurons for selecting two binary, mutually exclusive classes; it can be three neurons firing simultaneously with probabilities; it can be a hundred neurons with whatever activation the problem happens to demand! Architecting the output layer is a real art that depends on the problem at hand. I will leave it open and build concrete examples in the next chapter. One last word in case this slipped under the radar: the matrix $w_{\eta\mu}^{(2)}$ belongs to the second layer and it connects every μ -neuron from the previous layer to every η -neuron in the output layer; final answer $\hat{y}_\eta^i$ is a vector of dimension η coming from the i th data input.

A picture is worth more than a thousand words, isn't it?



For the sake of completeness, let me write one last formula for the loss minimization condition. With all poetic license permission from mathematics, I'll write it this way:

$$\frac{\partial}{\partial(w, b)} L = 0 \quad (3.5)$$

So much is hidden behind (3.5), but since for us it is a symbolic equation, we need not worry. Modern frameworks like PyTorch and TensorFlow take care of all the algebra. Just to give you an idea: a network with 8 input, 12 hidden and 5 output neurons has a total of 173 trainable w and b parameters, and therefore (3.5) leads to a system of 173 equations. Cool, huh? To be a bit more explicit, take a look at the index notation of the gradient descent equations:

$$\begin{aligned} w_{\mu\nu}^{(\ell)} &\rightarrow w_{\mu\nu}^{(\ell)} - \alpha \frac{\partial L}{\partial w_{\mu\nu}^{(\ell)}} \\ b_{\mu}^{(\ell)} &\rightarrow b_{\mu}^{(\ell)} - \alpha \frac{\partial L}{\partial b_{\mu}^{(\ell)}} \end{aligned} \quad (3.6)$$

All 173 updates of a (8, 12, 5) network are accounted for in these two lines. What about backpropagation's formulae using index notation? The error signals per layer, defined as¹

$$\delta_{\eta}^{(2)i} \equiv \frac{\partial L}{\partial v_{\eta}^i}, \quad \delta_{\mu}^{(1)i} \equiv \frac{\partial L}{\partial u_{\mu}^i}. \quad (3.7)$$

¹In Eq. (2.6), $\delta_i^{(2)}$ was defined as the partial derivative of the loss with respect to the output $\hat{y}_i$; that was the case because the activation of the output layer was the identity $\mathbb{1}$, and there was no pre-activation variable v_i (or, $\hat{y}_i = \mathbb{1}(v_i)$ if you will). However, the error signal is always defined as the partial derivative of the loss with respect to the pre-activation, like in (3.7).

leads, via the chain rule for partial derivatives and the definitions (3.1)–(3.4), to the following expressions:

$$\begin{aligned}\delta_{\eta}^{(2)i} &= \frac{\partial L}{\partial \hat{y}_{\kappa}^i} \frac{\partial \hat{y}_{\kappa}^i}{\partial v_{\eta}^i} \\ \delta_{\mu}^{(1)i} &= \delta_{\eta}^{(2)i} w_{\eta\nu}^{(2)} \frac{\partial z_{\nu}^i}{\partial u_{\mu}^i}\end{aligned}\tag{3.8}$$

I ensured that the cross-terms $\partial \hat{y}_{\kappa}^i / \partial v_{\eta}^i$ and $\partial z_{\nu}^i / \partial u_{\mu}^i$ were explicitly presented in Eq. (3.8). They are necessary when neurons have coupled activations—that is, when a neuron’s activation function depends on the activation functions of all other neurons in the same layer. This is the case, for example, when dealing with a classification problem: only one neuron should fire with a value of $\approx 100\%$, while the others remain at $\approx 0\%$. The neurons “communicate” with each other precisely through these cross-terms in the error signal. If, on the other hand, the activations act one neuron at a time, the cross-terms will be zero whenever $\kappa \neq \eta$ and $\nu \neq \mu$, and only the diagonal terms will survive.

The derivatives appearing in (3.6) can then be written as:

$$\begin{aligned}\frac{\partial L}{\partial w_{\eta\mu}^{(2)}} &= \sum_i \delta_{\eta}^{(2)i} z_{\mu}^i & \frac{\partial L}{\partial b_{\eta}^{(2)}} &= \sum_i \delta_{\eta}^{(2)i} \\ \frac{\partial L}{\partial w_{\mu\nu}^{(1)}} &= \sum_i \delta_{\mu}^{(1)i} x_{\nu}^i & \frac{\partial L}{\partial b_{\mu}^{(1)}} &= \sum_i \delta_{\mu}^{(1)i}\end{aligned}\tag{3.9}$$

Observe how (3.9) beautifully generalizes the backpropagation formulae we obtained for the two-neuron toy model in Eq. (2.8): it is the very same rule — error signal times the input arriving at that layer. Nothing changed but the presence of the indices.

With that, we have just covered the key theoretical results needed to build a truly realistic, multilayer model. So, let’s get to it!

Chapter 4

Concrete examples

Let's build our first model out of equations (3.1)—(3.6). For didactic purposes, I will use Keras to write this very first piece of code in the book:

```
1 import keras
2 from keras import layers
3
4 N_FEATURES = 8
5 N_HIDDEN = 12
6 N_CLASSES = 5
7
8 model = keras.Sequential(
9     [
10         keras.Input(shape=(N_FEATURES,)),
11         layers.Dense(N_HIDDEN, activation="relu"),
12         layers.Dense(N_CLASSES, activation="softmax")
13     ]
14 )
```

The above implementation faithfully represents the network depicted in the figure of the preceding Chapter: an input layer that receives vectors of dimension 8, followed by a hidden layer of 12 neurons with ReLU activation, and an output layer of 5 neurons with softmax activation.

The ReLU activation of the hidden layer, given by

$$\sigma_{\text{ReLU}}(x) = \max(0, x) \quad (4.1)$$

is a fairly standard choice in many modern applications (it simply returns x if $x > 0$ and 0 otherwise) due to its low computational cost and empirically proven efficiency. But, what about softmax activation in the output layer? Why did I choose it?

Softmax activation, given by

$$\hat{y}_\eta = \sigma_{\text{softmax}}(v_\eta) = \frac{e^{v_\eta}}{\sum_{\kappa=1}^5 e^{v_\kappa}} \quad (4.2)$$

has one very powerful property: the output of all 5 neurons sums to one, that is, $\sum_{\eta=1}^5 \hat{y}_\eta = 1$ (observe how all output neurons are coupled to each other through the denominator of (4.2)). Each output value $\hat{y}_\eta$, therefore, can be interpreted as the probability that input x_ν belongs to class η . Therefore, the choice of softmax activation in the output layer made this network suitable for handling classification problems: given a set of features¹ x_ν , the model answers “to which class η does this data point belong?”

¹The coordinates of a datapoint, that is, the information characterizing that datapoint, are called features.

To complete the model, the loss function and the learning rate must be chosen. In the first Chapter, we used the Mean Squared Error loss function from equation (1.3) to derive the backpropagation formulas, as it was the most intuitive loss function to introduce the topic in the context of fitting experimental data points. However, when a network employs softmax activation, the MSE loss function is no longer a suitable choice; instead, the Cross-Entropy loss function should be used:

$$L_{\text{CE}} = - \sum_{i=1}^N \sum_{\eta=1}^5 y_{\eta}^i \log(\hat{y}_{\eta}^i) \quad (4.3)$$

Had we used MSE loss,

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \sum_{\eta=1}^5 (y_{\eta}^i - \hat{y}_{\eta}^i)^2 \quad (4.4)$$

the error signals from (3.8) would read

$$\begin{aligned} \delta_{\eta}^{(2)i} &= -\frac{2}{N} \hat{y}_{\eta}^i \left[(y_{\eta}^i - \hat{y}_{\eta}^i) - \sum_{\kappa=1}^5 (y_{\kappa}^i - \hat{y}_{\kappa}^i) \hat{y}_{\kappa}^i \right] \\ \delta_{\mu}^{(1)i} &= \delta_{\eta}^{(2)i} w_{\eta\mu}^{(2)} \sigma^{(1)'}(u_{\mu}^i) \end{aligned} \quad (4.5)$$

which has the following problematic behaviour: the whole error signal carries an overall factor $\hat{y}_{\eta}^i$. If the model makes a confidently wrong prediction (e.g., target $y_{\eta}^i = 1$, but predicted $\hat{y}_{\eta}^i \approx 0$), the bracket remains of order one while that prefactor collapses. As a result, the entire gradient vanishes, making weight updates nearly zero when the error is at its largest. The network stops learning precisely when it needs to adjust the most².

If, instead, we use Cross-Entropy loss, the logarithm perfectly cancels out the softmax derivative, leading to the clean and simple error signals:

$$\begin{aligned} \delta_{\eta}^{(2)i} &= \hat{y}_{\eta}^i - y_{\eta}^i \\ \delta_{\mu}^{(1)i} &= (\hat{y}_{\eta}^i - y_{\eta}^i) w_{\eta\mu}^{(2)} \sigma^{(1)'}(u_{\mu}^i) \end{aligned} \quad (4.6)$$

The error signals derived from Eq. (4.6) are not only clean and simple but also exhibit notably desirable behaviour: the error is at its maximum precisely when the model is most mistaken. This behaviour steers the model's learning in the right direction. It is a classic example of the importance of using the appropriate combinations of loss function and activation function. Table 4.1 summarizes the most relevant known use cases.

The implementation of the cross-entropy loss function in Keras for our (8, 12, 5) network can be achieved with the following code snippet. The complete, functional code is available in the GitHub repository, alongside several other didactic examples covering different activation-loss configurations.

```
1 model.compile(
2     optimizer=keras.optimizers.SGD(learning_rate=0.01),
3     loss=keras.losses.SparseCategoricalCrossentropy(),
4     metrics=["accuracy"],
5 )
```

²This paragraph was written by Gemini and adapted to index notation by Claude. They gave such a clear explanation that I decided to Ctrl+C.

Table 4.1: Loss Functions and Output Layer Activations by Machine Learning Task

Problem Type	Network output	Activation	Loss Function
Binary Classification	One probability, $\hat{y} \in (0, 1)$	Sigmoid	Binary Cross-Entropy (Log Loss)
Multi-Class Classification	Coupled probabilities, $\sum_{\eta} \hat{y}_{\eta} = 1$	Softmax	Categorical Cross-Entropy
Multi-Label Classification	Independent probabilities, each $\hat{y}_{\eta} \in (0, 1)$	Sigmoid (element-wise)	Binary Cross-Entropy
Regression	Unbounded real value, $\hat{y} \in \mathbb{R}$	Linear (None)	Mean Squared Error (MSE)
Non-Negative Regression	Non-negative real value, $\hat{y} \geq 0$	ReLU or Softplus	MSE or MAPE
Bounded Regression	Bounded value, $\hat{y} \in (0, 1)$, rescaled to the desired range	Sigmoid	MSE or MAE

MSE: Mean Squared Error. MAE: Mean Absolute Error. MAPE: Mean Absolute Percentage Error.

The second column describes what the network *emits*, not the format in which the target is supplied: for multi-class problems the target y_{η}^i may be given either as a one-hot vector, as in (4.3), or as a single integer class index. The two are numerically equivalent, and frameworks provide a separate “sparse” entry point for the latter.

Compare the second and third rows, as they are the two faces of the discussion following (3.8): softmax couples the output neurons through a shared normalization, so its derivative carries the cross-terms, whereas an element-wise sigmoid leaves every neuron independent and its derivative is purely diagonal. Same number of neurons, same probabilities—but one layer converses with itself and the other does not.

Chapter 5

Towards advanced topics

All the theory presented so far forms a solid foundation for us to begin discussing more advanced topics. I would like to start by stating the

Universal Approximation Theorem: a feedforward neural network¹ with a single hidden layer containing a finite number of neurons can approximate any continuous function on compact subsets of $\mathbb{R}^n$ to any desired degree of accuracy, given appropriate non-linear activation functions.

Mathematically, a single-hidden-layer network outputs

$$f(x) = \sum_{i=1}^N v_i \sigma(w_i^T x + b_i) \quad (5.1)$$

where $w_i \in \mathbb{R}^n$ and $v_i, b_i \in \mathbb{R}$ are weights and biases, and σ is the activation function. The theorem guarantees that for any target continuous function $F : K \rightarrow \mathbb{R}$ on a compact set $K \subset \mathbb{R}^n$ and any arbitrary precision $\varepsilon > 0$, there exists a network $f(x)$ such that

$$\sup_{x \in K} |f(x) - F(x)| < \varepsilon \quad (5.2)$$

The paragraph above is a bit more mathematical than the rest of the book used to be—in fact, I copied and pasted it from Gemini—but a little calculus will enable you to understand it thoroughly. Recall Chapter 1, in which we first introduced a neural network as the fitting of a function to data. Now, imagine a multidimensional space filled with data points; suppose there exists a network, $F(x)$, that fits all this data. Then the theorem guarantees that there always exists a function $f(x)$, in the form of Eq. (5.1), capable of approximating $F(x)$ to any desired precision ε .

So, can we now solve any problem with the network from Chapter 3? Do we simply insert an arbitrary number of neurons in the hidden layer and... well, not so fast! The theorem guarantees the existence of such a network, but never provides the recipe for finding its weights and bias. Furthermore, a “finite number” of neurons can be something frighteningly large, and the more neurons there are, the harder convergence becomes, due to peaks and valleys in the loss surface and computational cost. This is where our advanced topics begin: we will explore techniques that allow us to find $f(x)$ efficiently in order to solve specific problems.

¹A feedforward neural network is precisely what we built in Chapter 3: a network in which information flows in a single direction, layer by layer, without cycles or feedback. One point worth noting: the backpropagation of Eq. (3.8) does indeed propagate error signals in the reverse direction; however, this does not cause the network to cease being feedforward. The term feedforward actually refers to the flow $x \mapsto \hat{y}$.

Chapter 6

Appendix A

- *ReLU* (Rectified Linear Unit), $\text{ReLU}(x) = \max(0, x)$ — the default choice: trivially cheap to evaluate and it never saturates for $x > 0$, so the error signal survives many layers.
- *Leaky ReLU*, $\text{LReLU}(x) = \max(\gamma x, x)$ with a small fixed γ (typically 0.01) — cures the “dead neuron” pathology of ReLU, whose derivative vanishes identically for negative inputs.
- *PReLU* (Parametric ReLU), same form as Leaky ReLU but with γ promoted to a trainable parameter — lets the network learn how leaky it wants to be, at the cost of one extra parameter per neuron (or per layer).
- *ELU* (Exponential Linear Unit),

$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \beta(e^x - 1), & x \leq 0 \end{cases}$$

— smooth everywhere and saturating at $-\beta$ for large negative inputs, which pulls the mean activation toward zero and speeds up convergence.

- *GELU* (Gaussian Error Linear Unit), $\text{GELU}(x) = x \Phi(x) = \frac{x}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$, with Φ the standard normal cumulative distribution — a smooth, probabilistically motivated gate that weights each input by how likely it is to be positive; the standard in modern transformer architectures.
- *Sigmoid/Logistic*, $\sigma(x) = 1/(1 + e^{-x})$ — our old friend from chapter 2, squashing the real line into $(0, 1)$; ideal for probabilities at the output, but its derivative never exceeds $1/4$, which starves deep networks of gradient.
- *Tanh*, $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ — a rescaled sigmoid mapping onto $(-1, 1)$, preferable to it because it is centred at zero, though it saturates just as badly.